

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Микропроцессорные техника в мехатронике и
роботехнике»
ТЕМА: Работа с серомашинками (ШИМ + регистр ICR)
Вариант 8

Студенты гр. 2491

Преподаватель



Сулинский Е.В.
Недовизий А.И.

Копычев М.М.

Санкт-Петербург

2025

Задание на лабораторную работу

Написать программу управления двумя сервомашинками, причем одна сервомашинка должна управляться с помощью ползункового потенциометра, а вторая – с энкодера.

Основные теоретические положения

Сервомашинки или сервоприводы нашли широкое применение в области роботостроения и моделизме. Большинство сервомашин используют три провода для работы. Первый провод – для питания, обычно 4.8 В или 6 В, второй – общий провод или – земля и третий – сигнальный провод. Управляющий сигнал передаёт информацию о требуемом положении выходного вала. Вал связан с потенциометром, который определяет его положение. Контроллер в сервоприводе по сопротивлению потенциометра и значению управляющего сигнала определяет, в какую сторону требуется вращать ротор, чтобы получить нужное положение выходного вала. На рисунке 1 приведена схема внутреннего устройства сервомашинки.



Рисунок 1 – Устройство сервомашинки

Для управления сервоприводом потребуется формировать ШИМ с частотой 50 Гц. При этом для одного крайнего положения длительность импульса должна составлять 1 мс, а для другого положения – 2 мс. На рисунке 2 приведено графическое представление управляющего сигнала.

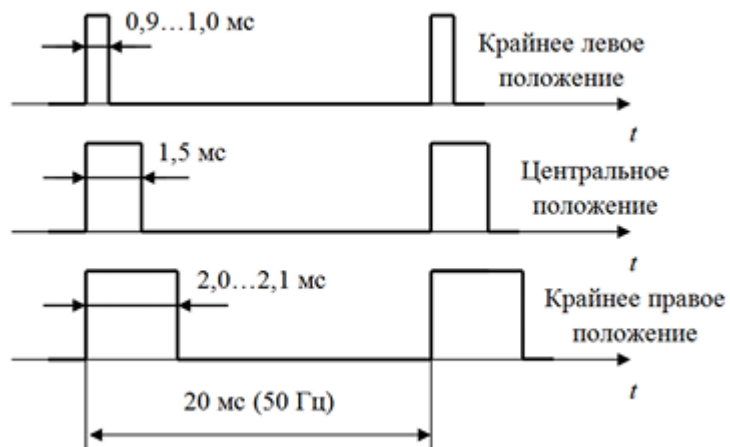


Рисунок 2 – Временная диаграмма управляющего сигнала

Крайним положениям сервомашинки соответствуют углы $+45^\circ$ и -45° относительно нулевого положения (для 90-градусной сервомашинки). На рисунке 3 приведена схема подключения сервомашинок.

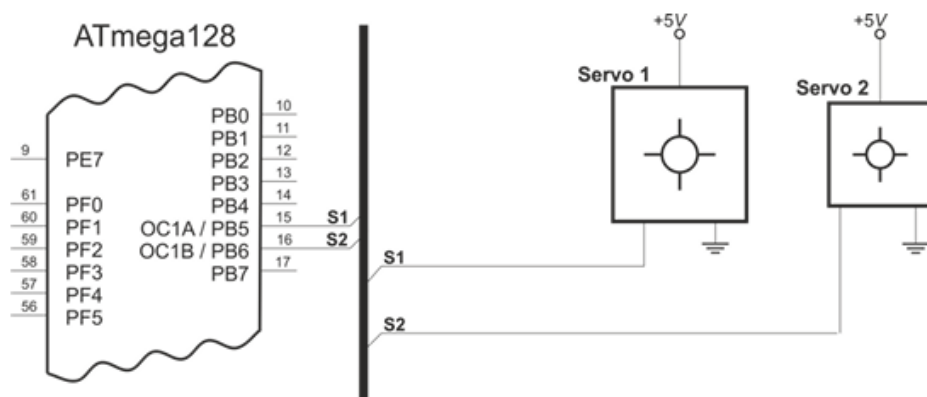


Рисунок 3 – Схема для лабораторной работы

Блок-схемы

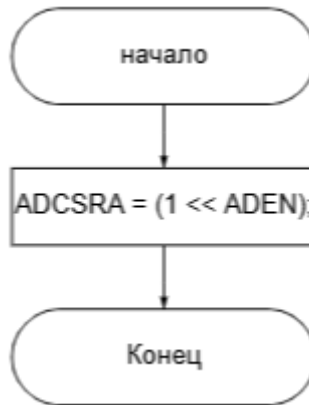


Рисунок 4 – Блок-схема функции `adcInit()`

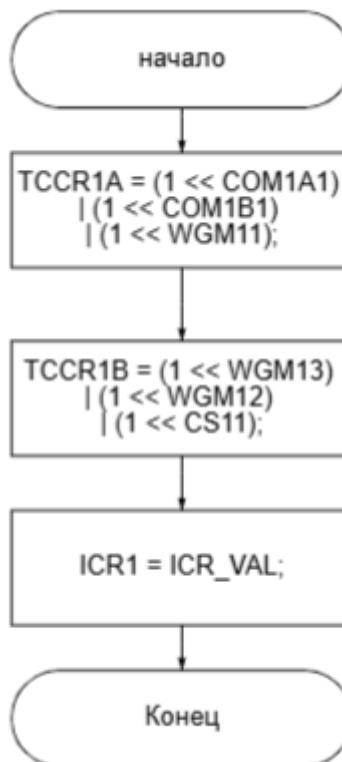


Рисунок 5 – Блок-схема функции `timerInit()`

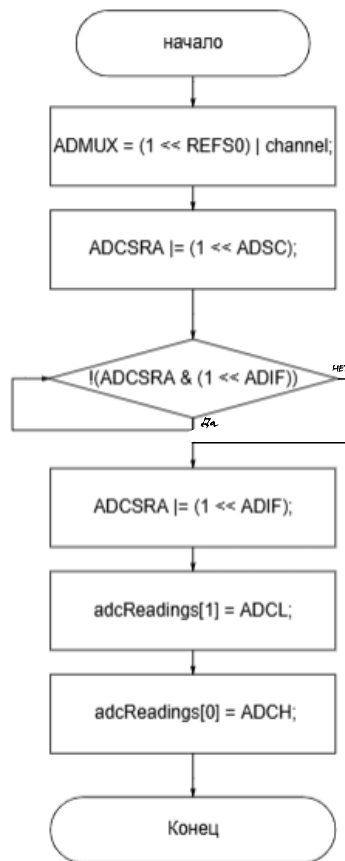


Рисунок 6 – Блок-схема функции `updateAdcReading(uint8_t channel)`

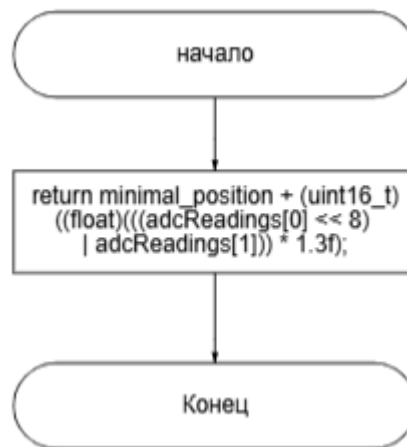


Рисунок 7 – Блок-схема функции `convertAdcReadings(uint16_t minimal_position)`

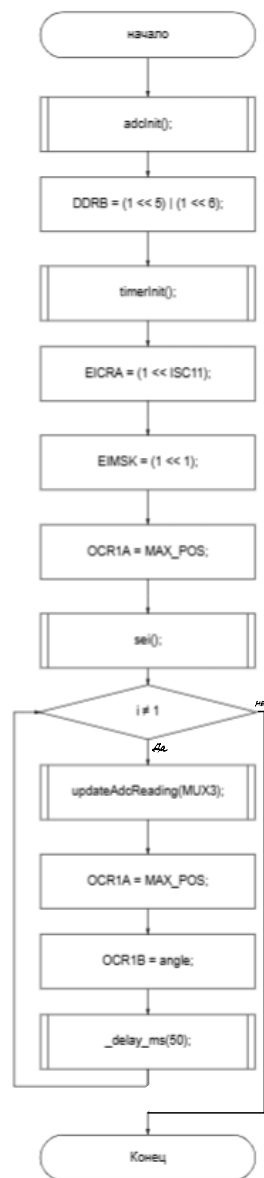


Рисунок 8 – Блок-схема функции main(void)

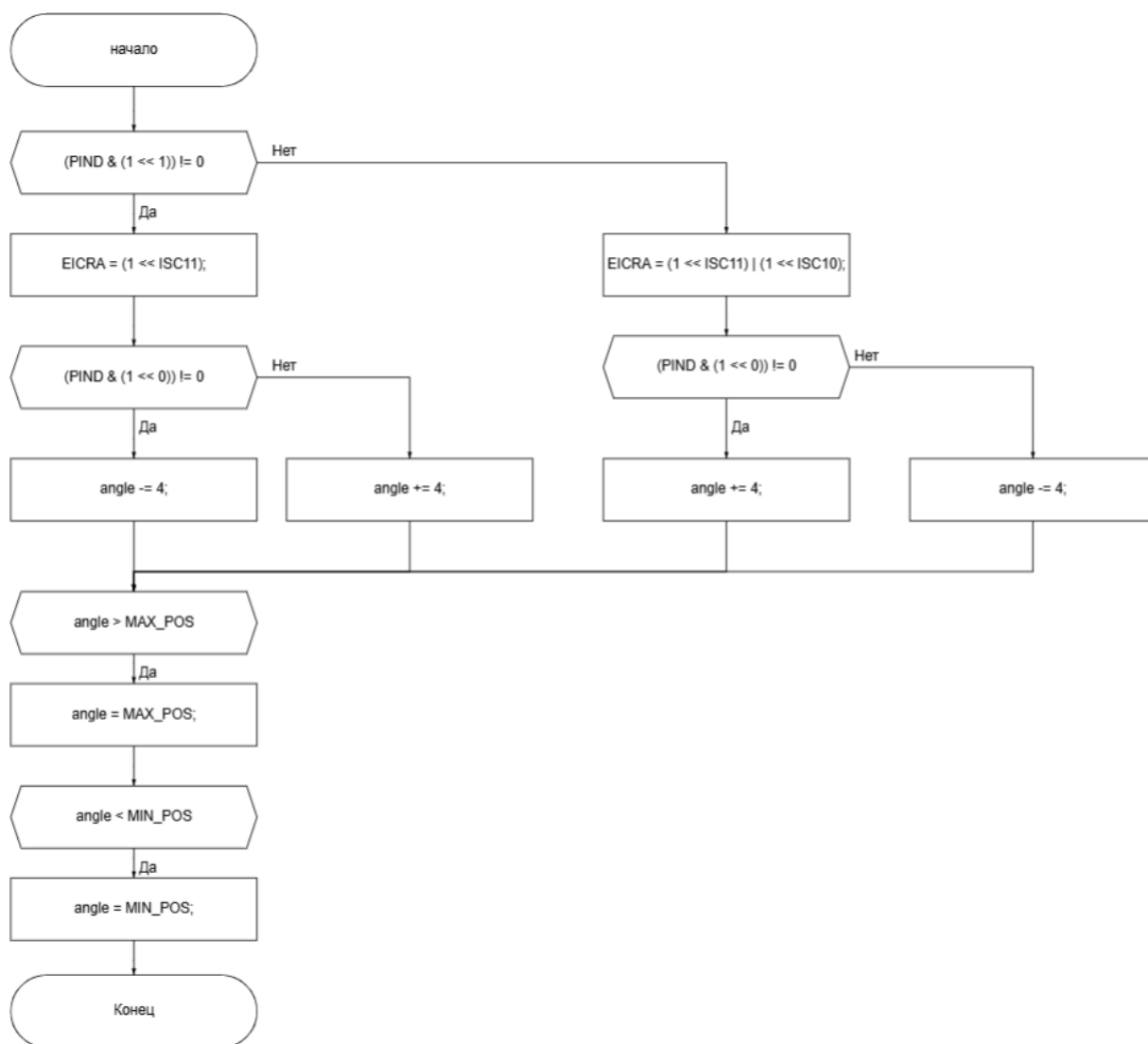


Рисунок 9 – Блок-схема функции ISR(INT1_vect)

ВЫВОДЫ

В ходе выполнения лабораторной работы №4 была успешно разработана и реализована система управления сервомашинками с использованием двух различных методов управления: через механический энкодер и аналоговый потенциометр. Программа демонстрирует практическое применение 16-битного таймера/счетчика №1 в режиме быстрой ШИМ с переменным верхним пределом, задаваемым через регистр ICR1, что является оптимальным решением для генерации управляющих импульсов сервоприводов с требуемой частотой 50 Гц.

Ключевым достижением работы стала корректная настройка таймера для формирования ШИМ-сигнала с точными временными параметрами. Расчетное значение $ICR1 = 27648$ при предделителе на 8 обеспечивает частоту 50 Гц, соответствующую стандарту управления сервомашинками. Установка регистров сравнения OCR1A и OCR1B в диапазоне от MIN_POS (1382) до MAX_POS (2765) позволяет точно позиционировать сервы в пределах их рабочего угла, где минимальное значение соответствует импульсу длительностью 1 мс, максимальное - 2 мс, а среднее положение - 1.5 мс.

Особое внимание было уделено обработке сигналов от инкрементного энкодера через прерывание INT0. Реализованный алгоритм динамической перенастройки чувствительности прерывания обеспечивает надежное определение направления вращения энкодера по фазовому сдвигу между каналами A и B. Ограничение переменной $encK$ в диапазоне 0-255 с последующим преобразованием в позицию сервомашинки через линейное масштабирование позволяет осуществлять плавное и предсказуемое управление.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#define ICR_VAL 27648
#define MIN_POS (ICR_VAL / 20)
#define MAX_POS (ICR_VAL / 10)

uint16_t angle = (MAX_POS + MIN_POS) / 2;
uint8_t adcReadings[2] = {0, 0};
void adcInit()
{
    ADCSRA = (1 << ADEN); // включение АЦП
}

void timerInit()
{
    // быстрая ШИМ, предел счёта: ICR, предделитель: 8
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);
    ICR1 = ICR_VAL;
}

void updateAdcReading(uint8_t channel)
{
    // AVCC + правое выравнивание + выбор канала
    ADMUX = (1 << REFS0) | channel;
    ADCSRA |= (1 << ADSC); // Начало преобразования
    // Ждем окончания преобразования АЦП
    while (!(ADCSRA & (1 << ADIF)));
    // Сброс флага завершения преобразования
    ADCSRA |= (1 << ADIF);
    adcReadings[1] = ADCL;
    adcReadings[0] = ADCH; // возвращаем 10 бит результата
}
```

```

uint16_t convertAdcReadings(uint16_t minimal_position)
{
    return minimal_position + (uint16_t)((float)(((adcReadings[0]
<< 8) | adcReadings[1])) * 1.3f);
}

int main(void)
{
    adcInit();
    DDRB = (1 << 5) | (1 << 6);
    timerInit();
    EICRA = (1 << ISC11);
    EIMSK = (1 << 1);
    OCR1A = MAX_POS;
    sei();
    while (1)
    {
        updateAdcReading(MUX3); // чтение порта 3 канала АЦП
        OCR1A = convertAdcReadings(MIN_POS);
        OCR1B = angle; // this servo is broken so it won't do
shit :(
        _delay_ms(50);
    }
}

ISR(INT1_vect)
{
    if ((PIND & (1 << 1)) != 0)
    {
        EICRA = (1 << ISC11);
        if ((PIND & (1 << 0)) != 0)
            angle -= 4;
        else
            angle += 4;
    }
    else

```

```

{
    EICRA = (1 << ISC11) | (1 << ISC10);
    if ((PIND & (1 << 0)) != 0)
        angle += 4;
    else
        angle -= 4;
}

if (angle > MAX_POS)
    angle = MAX_POS;
if (angle < MIN_POS)
    angle = MIN_POS;
}

```